A Reproducible Local-Ratio and SCC-Refinement Framework for Weighted Ordering in Directed Graphs

Soroush Vahidi[a]

[a]*Department of Computer Science, New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

Weighted directed graphs model precedence conflicts, cyclic dependencies, and pairwise ordering data in engineering and decision systems. This paper studies the minimum weighted feedback arc set problem on nonnegative weighted directed graphs, equivalently the problem of finding an ordering with minimum total backward edge weight. We present a reproducible heuristic framework that combines an engineered local-ratio cycle-reduction method with topological add-back, a weighted seed construction, and incumbent-protected refinement over strongly connected components. The refinement improves local cyclic substructures while preserving a nonworsening invariant relative to the best seed solution.

The computational study combines sparse public graph benchmarks, an ablation study, exact validation on small instances, external baseline comparisons, and a dense linear-ordering transfer test. On the exact-validation subset, where bitmask dynamic programming certifies optima, the proposed refinement matches the optimum on 56 of 57 standard instances, with a mean relative gap of about 0.0006 percent. On 97 standard nonnegative sparse benchmark instances, it attains the best observed backward weight among the tested methods on 96 instances and outperforms all tested baselines, including the strongest external heuristic. For instances outside the exact-validation subset, "best" refers to the minimum backward weight observed among the compared algorithms, not a proof of optimality. Dense

*Corresponding author

Email address: sv96@njit.edu (Soroush Vahidi)

LOLIB instances are analyzed separately as a transfer regime; a matrix-based pairwise-ordering external method is stronger on that benchmark, while the proposed framework remains a general weighted-digraph heuristic.

Keywords: Minimum weighted feedback arc set, weighted ordering, local-ratio algorithm, strongly connected components, incumbent-protected refinement, linear ordering problem, computational heuristics

---

1. Introduction

Many engineering and decision systems require a linear order to be extracted from directed, weighted, and internally inconsistent information. A precedence graph may encode that task u should precede task v, that a circuit signal should be routed before a dependent component is evaluated, that a package should be installed before another package can be configured, or that an alternative is preferred to another alternative in paired-comparison data. Rank aggregation from inconsistent pairwise information is a standard motivation for ordering heuristics Ailon et al. (2008). In practice these directed relations are often cyclic. Cycles can arise from noisy measurements, conflicting expert judgments, reciprocal dependencies, approximate model reductions, or competing design objectives. A useful ordering method must therefore decide which directed constraints to respect and which constraints to place backward in the final order. When arcs carry weights, the backward constraints are not interchangeable: violating a high-weight precedence relation should be more costly than violating a weak or noisy relation. This leads naturally to the minimum weighted feedback arc set (MWFAS) problem, a weighted ordering problem on directed graphs with direct relevance to cyclic-dependency resolution, ranking, scheduling, and other computational decision workflows in industrial engineering.

Informally, the minimum weighted feedback arc set problem asks for a minimum-weight set of arcs whose removal makes a directed graph acyclic. Equivalently, it asks for a vertex ordering that minimizes the total weight of arcs directed backward with respect to that order. The unweighted problem is NP-hard in general Karp (1972), and weighted variants have a long heuristic and exact-method literature Flood (1990); Baharev et al. (2021). Exact methods are practical only for small instances or special structures. Naive score heuristics can be very fast, but they may ignore local cyclic structure; random multistart heuristics can provide a broad baseline, but they do not

exploit graph decomposition; and specialized dense-ordering solvers may perform well on complete pairwise-comparison instances while being less directly aligned with sparse, irregular directed graphs. The computational question addressed here is not whether one heuristic dominates every possible ordering instance. Instead, it is whether a carefully engineered and reproducible framework can deliver strong performance on nonnegative sparse weighted digraphs while making its scope limits explicit.

The local-ratio perspective is an important starting point for feedback arc set heuristics and approximation-oriented reasoning. We do not claim local-ratio as new. Broader local-ratio foundations appear in Bar-Yehuda et al. (1998), and directed feedback problems were treated algorithmically in Demetrescu and Finocchi (2003). The contribution in this paper is algorithm-engineering rather than a new approximation theorem: we study a reproducible local-ratio cycle-reduction implementation that records cycle-removal decisions, reconstructs an acyclic topological order, and then performs a topological add-back phase to recover arcs whenever doing so does not reintroduce cyclicity. This method is referred to as local-ratio topological add-back (LR-TA) after the components are defined in Section 4. The emphasis is on deterministic data handling, measurable component effects, and integration with a refinement stage that is evaluated against exact and external baselines.

The integrated framework combines three components. First, the local-ratio topological add-back procedure produces a feasible ordering from weighted cycle reductions and a reconstruction step. Second, a WMSF-style weighted seed construction provides a complementary initial ordering and internal baseline. Third, incumbent-protected refinement searches within strongly connected components (SCCs). We refer to this refinement as incumbent-protected SCC neighborhood search (IPSNS). IPSNS updates the incumbent only when a candidate ordering has strictly smaller backward weight. This protection gives a simple invariant: the final solution cannot be worse than the best internal seed under the same objective and nonnegative-weight assumptions. The invariant is not an approximation guarantee, but it prevents refinement from degrading the seed and makes the local search behavior traceable instance by instance.

The computational study is organized around five evidence streams. The main sparse benchmark evaluates public weighted directed graph instances derived from graph-benchmark collections, with negative-weight instances separated from the standard nonnegative comparisons. An ablation study isolates the effect of the topological add-back phase, the use of the best

seed, the incumbent-protected refinement, and the SCC-priority choice on a representative subset. An exact small-instance validation compares heuristic orderings against a bitmask dynamic-programming optimum where exact optimization is feasible. A sparse external-baseline study compares the proposed framework with the open-source DRMacIver/FAS heuristic, the Eades heuristic as implemented in python-igraph, a weighted adaptation of Eades et al. Eades et al. (1993), a Borda net-score ordering, and random multistart. Finally, a dense transfer study evaluates whether the framework carries over to complete weighted linear-ordering instances from the Linear Ordering Problem Library (LOLIB).

The results support a deliberately bounded claim. On 105 sparse benchmark instances, the incumbent-protected refinement has no incumbent violations relative to either internal seed. In the ablation study, the add-back phase reduces mean backward weight by 5.9 percent relative to local-ratio without add-back, and the refinement gives an additional 0.75 percent mean reduction on the tested subset. On the exact-validation subset of 57 standard nonnegative instances with $n \leq 20$, the refinement matches the dynamic-programming optimum on 56 instances, with mean relative gap about 0.0006 percent. On 97 standard nonnegative sparse instances in the external comparison, it achieves the best observed backward weight among the tested methods on 96 instances; the strongest external baseline, DRMacIver, is about 21.6 percent worse in mean backward weight on this sparse benchmark. These numbers support strong performance on sparse nonnegative weighted digraphs, but they do not support a universal dominance claim.

The dense LOLIB study is reported as a scope boundary rather than as primary evidence. LOLIB instances are complete weighted ordering instances, closely related to the linear ordering problem and structurally different from the sparse directed graphs that motivate the framework. On 50 dense LOLIB instances, the matrix-based pairwise-ordering baseline DRMacIver/FAS obtains the best observed solution on 45 instances, while the proposed refinement is best on 5 instances and has about 3.88 percent higher mean backward weight. This outcome is consistent with the structural distinction: SCC-local refinement is useful when sparse cyclic subgraphs provide meaningful neighborhoods, whereas dense complete ordering instances favor matrix-based pairwise-ordering heuristics. The method is therefore positioned as a reproducible general weighted-digraph framework with strong sparse performance and an explicitly documented dense-ordering limitation.

The contributions of this paper are as follows:

- We present an engineered local-ratio topological add-back heuristic for nonnegative weighted feedback arc set instances, separating inherited local-ratio ideas from the implementation and add-back choices studied here.
- We introduce incumbent-protected SCC neighborhood refinement, a local-search framework that preserves non-worsening relative to the best internal seed and can be checked for incumbent violations.
- We provide a multi-part computational evaluation covering sparse public benchmarks, component ablation, exact small-instance validation, external sparse baselines, and dense LOLIB transfer testing.
- We state explicit claim boundaries for negative-weight exclusions, dense linear-ordering instances, weighted Eades adaptation, reproducibility, and the absence of a new approximation guarantee.

The remainder of the paper is organized as follows. Section 2 reviews feedback arc set heuristics, local-ratio methods, ordering heuristics, exact methods, and linear-ordering benchmarks. Section 3 defines the minimum weighted feedback arc set objective, the ordering-induced backward weight, and the relation to dense linear-ordering notation. Section 4 describes the local-ratio topological add-back procedure, the weighted seed, and the incumbent-protected SCC refinement. Section 5 presents datasets, baselines, metrics, and reproducibility controls. Section 6 summarizes the computational results. Section 7 discusses limitations, including the dense LOLIB boundary, and Section 8 concludes.

2. Related work

2.1. Feedback arc set and local-ratio methods

The feedback arc set problem and its weighted variants sit at the intersection of graph optimization, ordering, and ranking. In directed graphs, the problem can be phrased either as deleting a minimum-weight set of arcs to obtain an acyclic graph or as finding an ordering with minimum backward weight. That equivalence makes the problem relevant both to graph-theoretic cycle breaking and to applications that begin from pairwise precedence or preference data. Classical digraph formulations identified minimum feedback arc sets as a central cyclic-structure question Lempel and Cederbaum

(1966), early weighted treatments framed the problem as an ordering objective Flood (1990), and the general decision problem is computationally hard Karp (1972). The approximation literature is broader for vertex-deletion or undirected variants than for the weighted directed setting considered here, but it still provides important methodological context. Even et al. (1998) study approximation algorithms for minimum feedback sets and multicuts in directed graphs, while Hecht et al. (2021) analyze tight localizations of feedback sets in more recent algorithm-engineering work. In particular, Bar-Yehuda et al. (1998) place local-ratio reasoning in a general approximation framework, showing how weighted combinatorial objectives can be decomposed while preserving feasible improvement structure. Their setting is not the same as minimum weighted feedback arc set on directed graphs, and we do not rely on their theorem to obtain a new approximation guarantee here. However, the paper is directly relevant to the design logic of iterative weight reductions on conflict structures.

For directed feedback problems, Demetrescu and Finocchi (2003) provide the closest methodological antecedent to the cycle-reduction component used here. Their contribution is important for two reasons. First, it treats directed feedback problems algorithmically rather than only as ordering formulations. Second, it shows how local-ratio reductions can be applied to directed cycles in a way that is compatible with practical heuristics. The local-ratio step in the present work is inherited in spirit from that line of research: when a directed cycle is found, all arcs on the cycle are reduced by the minimum weight on the cycle, at least one arc becomes tight, and the active graph is simplified. What is new here is not the local-ratio principle itself, but an engineered sparse-graph implementation that records removed arcs explicitly, reconstructs a deterministic topological order once the active graph becomes acyclic, and then runs a heavy-first add-back step to reduce unnecessary deletions.

The contribution is therefore best understood as an algorithm-engineering framework built around a known reduction principle. The emphasis is on deterministic data handling, compatibility with sparse weighted digraphs, and integration with a second-stage refinement process that improves only when the global objective decreases.

2.2. Heuristics and software baselines

Heuristic work on feedback arc set and related ordering tasks spans greedy graph rules, local search, and software packages that expose ordering or arc-

deletion primitives. A classical baseline is the greedy heuristic of Eades et al. (1993). Their method is attractive because it is fast, interpretable, and structurally simple: it repeatedly extracts sources, sinks, or high net-score vertices to construct an order. An earlier denser-graph variant in the same heuristic family appears in Eades and Lin (1995). Even when a modern study ultimately outperforms such a heuristic, it remains an essential reference point because many later practical systems either implement that rule directly or build on the same score-based intuition. In the present paper, Eades-style ideas appear in two places: as the historical baseline that motivates fast greedy ordering, and as the conceptual starting point for external and adapted software baselines used in the experiments.

The external-baseline design intentionally mixes library-grade and specialized open-source tools. The python-igraph documentation for `Graph.feedback_arc_set` python-igraph developers (2026) exposes both an Eades-style heuristic and exact integer-programming modes. It is a useful baseline because it reflects what a practitioner could call from a general-purpose graph library. In contrast, the DRMacIver/FAS tool DRMacIver (2026) is a matrix-based pairwise-ordering heuristic rather than a general graph-analysis package. Its documentation formulates the input as a nonnegative $n \times n$ matrix W, where W_{ij} represents evidence that item i should precede item j, and seeks a permutation maximizing the total forward weight $\sum_{p_i<p_j} W_{ij}$. The repository describes the method as deterministic and reports a local-optimality property with respect to single-element moves; it does not present the tool as an exact optimizer. Because the documented model is matrix-based and has $O(n^2)$ complexity in the number of items, it is especially natural as a dense pairwise-ordering baseline, while remaining usable in our experiments through the sparse entry-list interface. Empirically, it is the strongest external baseline on the sparse standard benchmark and the dominant method on the dense LOLIB transfer test.

The remaining baselines in the experiments are deliberately simple and interpretable. Weighted net-score ordering and random multistart are not intended to represent state of the art; they provide calibration points. Net-score and win-based tournament rules are classical ranking ideas with theoretical support in the weighted tournament literature Coppersmith et al. (2010). The weighted Eades adaptation used here should also be interpreted carefully. The original Eades–Lin–Smyth paper Eades et al. (1993) does not define the weighted adaptation used in this study, so that baseline is described as a local adaptation rather than as the original authors' algorithm. That distinction

is part of the methodological hygiene of the study: external baselines should be credited as external when they are external, and local adaptations should be labeled as such even when they are inspired by well-known methods.

There is a broader family of learning-based ranking methods that could be discussed without entering the experimental comparison set. GNNRank He et al. (2022), for example, frames ranking from pairwise comparisons as a trainable graph-neural ranking problem. It is relevant as a signal that the ranking literature has moved beyond purely combinatorial heuristics, but it is not used as a baseline here because the present study focuses on deterministic, training-free graph heuristics rather than learned ranking models. Recent weighted feedback arc set heuristics such as the minimal-and-stable construction of Cavallaro and Cutello Cavallaro and Cutello (2025) are likewise outside the comparison set because they were not rerun for this study, but they illustrate continued heuristic interest in weighted feedback arc deletion.

2.3. Exact methods and ordering formulations

The exact side of the literature serves two different roles in this paper. First, it sets expectations about computational hardness. Second, it provides the reference standard for the small-instance validation experiment. Baharev et al. (2021) give a modern exact treatment of minimum feedback arc set through formulations that connect the problem to broader combinatorial optimization machinery. Their work is not the computational baseline for the full sparse benchmark in this paper; the sparse benchmark is too large for exact optimization to be the main comparison mode. Instead, exact methods enter the study through a limited validation regime on small instances, where the question is not whether exact optimization scales globally, but whether the heuristic framework is near-optimal when exact answers are available.

This distinction also helps separate feedback arc set from the larger family of ordering models. Dense linear ordering formulations are often written as forward-weight maximization on complete matrices, whereas sparse directed graph formulations arise more naturally as backward-weight minimization on incomplete digraphs. The two are mathematically linked, but they are not computationally identical in practice.

The present study deliberately moves between the two views. The sparse benchmark is the main claim-bearing environment and is presented in graph terms. The LOLIB transfer test is presented in linear-ordering terms because

those instances are dense complete ordering problems. Keeping both formulations visible is important. Otherwise the reader could mistakenly treat good sparse-graph performance as evidence of dense linear-ordering dominance, which the experiments do not support.

At larger scales, exact computation is often replaced by decomposition, approximation, or specialized engineering. Simpson et al. (2016) provide one useful point of reference by treating feedback arc set computation in a web-scale setting. Tournament and dense-ordering settings have their own fast heuristic literature, including local-search methods for weighted tournament feedback arc set Fomin et al. (2010) and fixed-parameter approaches for the unweighted tournament case Alon et al. (2009). Their work is not directly comparable to the exact small-instance validation in this paper, but it illustrates the broader computational reality: scalable feedback arc methods are usually driven by structure, heuristics, or deployment constraints rather than by globally exact optimization. The goal here is not to displace exact methods; it is to present a heuristic framework whose computational behavior is strong on the sparse benchmark and whose limitations on dense complete ordering instances are reported explicitly.

2.4. Linear ordering benchmarks and positioning

The dense-transfer side of the paper is best understood through the linear ordering literature. Marti et al. (2012) provide a benchmark-oriented account of the Linear Ordering Problem Library (LOLIB) and its heuristic evaluation culture. That paper is the most appropriate formal reference for LOLIB-style benchmark usage here because it anchors the library in a published benchmark-comparison setting rather than only as a download page. The official LOLIB page itself Marti (2010) remains important for provenance and dataset identification, especially because benchmark users often obtain the instances directly from library pages or their mirrors. Together, these sources explain why dense complete tournaments have a different methodological status from sparse DIMACS-style directed graphs: they are not merely larger graphs of the same type, but instances drawn from a different benchmark tradition with different algorithmic expectations.

The sparse benchmark side has a different provenance. The graph-benchmarks collection Ali Dasdan (2026) is a public set of directed instances in DIMACS-style format rather than a linear-ordering library.

That difference supports a central design choice: the sparse and dense benchmarks test different claims. The sparse benchmark is the main evidence

for practical value on nonnegative weighted digraphs. The dense benchmark is a transfer test used to determine where a general weighted-digraph heuristic stops being competitive against dense-ordering-native methods.

The ranking-from-pairwise-comparisons viewpoint connects these benchmark families conceptually. In dense complete settings, every ordered pair carries evidence, so forward-weight maximization aligns naturally with tournament and ranking formulations Ailon et al. (2008); Coppersmith et al. (2010). In sparse settings, many pairwise relations are absent, heterogeneous, or structurally constrained by applications such as dependency repair or precedence resolution.

This is exactly why a method that performs well on sparse graph-benchmarks may still lose on LOLIB. The issue is not inconsistency in the experiments; it is a change in problem structure. Related work therefore has to prepare the reader for a comparative message that is strong but bounded.

This paper occupies a specific position in the literature. It is not a new approximation-theory paper in the style of local-ratio foundations Bar-Yehuda et al. (1998) or directed-feedback algorithm design Demetrescu and Finocchi (2003); Even et al. (1998). It is not an exact-method paper in the style of Baharev et al. (2021). It is not a web-scale systems paper in the style of Simpson et al. (2016). It is also not a dense linear-ordering or trainable ranking paper in the style of the LOLIB benchmark tradition Marti et al. (2012), tournament local search Fomin et al. (2010), or graph-neural ranking work such as GNNRank He et al. (2022). Instead, it is an integrated computational heuristic study whose main object is a reproducible framework for nonnegative weighted directed graphs.

That framework combines three roles that are often separated in the literature: a cycle-reduction seed grounded in prior local-ratio ideas, a complementary weighted seed, and incumbent-protected SCC neighborhood refinement. IPSNS is the main algorithmic contribution beyond the engineered LR-TA seed. It concentrates search on strongly connected components with positive backward contribution, applies SCC-local destroy-and-repair moves with restricted local-ratio repair, and accepts a candidate only when the global backward weight strictly decreases. The practical significance of that combination is twofold. First, it gives a transparent explanation for why the method should be robust on sparse directed graphs: the initial seeds construct feasible orders in different ways, and the refinement concentrates effort on strongly connected regions where ordering uncertainty remains. Second, it yields a claim that is strong enough to matter and narrow enough

to defend: the framework is highly effective on the main sparse benchmark, near-optimal on the exact-validation subset, and explicitly limited on dense LOLIB instances where the external matrix-based pairwise-ordering baseline is stronger.

3. Problem definition

Let $G = (V, A, w)$ be a directed graph with vertex set V, arc set $A \subseteq V \times V$, and nonnegative arc weights $w : A \to \mathbb{R}_{\geq 0}$. A feedback arc set is a set $F \subseteq A$ such that removing F makes the graph acyclic. The minimum weighted feedback arc set problem asks for a minimum-weight feedback arc set, i.e., a set F minimizing $\sum_{a \in F} w(a)$.

An ordering π assigns each vertex a distinct position. An arc (u, v) is backward under π if u appears after v. The ordering-induced backward weight is

$$\mathrm{bw}(\pi) = \sum_{(u,v) \in A:\ \pi(u) > \pi(v)} w(u, v). \tag{1}$$

For any ordering π, the set of backward arcs is a feedback arc set: all remaining forward arcs point from earlier to later positions and therefore form an acyclic graph. Conversely, every acyclic subgraph admits a topological order. The weighted feedback arc set problem can therefore be treated as the problem of finding an ordering with minimum backward weight Flood (1990); Karp (1972).

Dense linear ordering problem benchmarks are often stated in terms of a complete weight matrix C, where an ordering $\pi = (\pi_1, \ldots, \pi_n)$ is evaluated by the forward weight. In that notation,

$$\mathrm{fw}_C(\pi) = \sum_{1 \leq i < j \leq n} C_{\pi_i, \pi_j}, \qquad \mathrm{bw}_C(\pi) = \sum_{1 \leq i < j \leq n} C_{\pi_j, \pi_i} = W_C - \mathrm{fw}_C(\pi), \tag{2}$$

where $W_C = \sum_{i \neq j} C_{ij}$ is the total off-diagonal weight. Thus maximizing forward weight on a complete dense linear-ordering instance is equivalent to minimizing backward weight after converting the matrix to a complete weighted digraph. LOLIB results are reported as a transfer test. Those instances are dense complete ordering problems rather than sparse directed graph benchmarks.

Table 1: Algorithm components and their roles in the framework.

Component	Primary mechanism	Role in this paper
LR-TA	Global local-ratio cycle reduction followed by heavy-first add-back	Engineered reproducible seed; main constructive method derived from prior local-ratio ideas
WMSFstyle seed	Ordered arc removal, minimization, stabilization, and remimization inside SCCs	Complementary internal seed and baseline; broadens incumbent quality before refinement
IPSNS	SCC-local destroy–repair moves with restricted local-ratio repair and restricted add-back	Main refinement framework; improves only when the global objective decreases

4. Proposed methodology

The framework combines three layers: a local-ratio topological add-back seed, a complementary weighted seed, and an incumbent-protected refinement over strongly connected components. Figure 1 summarizes the data flow, while Tables 1 and 2 isolate the roles of each component and the invariants that the implementation preserves.

The implementation reads an aggregated weighted DIMACS digraph and stores it in edge-indexed arrays. Each arc receives an edge identifier together with fixed endpoint arrays and an activity flag. This design is shared across LR-TA, WMSF-style seeding, and IPSNS. The common objective is the backward weight defined in Section 3; all moves are evaluated under that objective, and all determinism claims in this section refer to fixed input graphs and fixed random seeds.

4.1. Local-ratio topological add-back

LR-TA is the first constructive component. It inherits the idea of local-ratio cycle reduction from the directed-feedback setting of Demetrescu and Finocchi (2003), but the implementation studied here is an engineered sparse-graph instantiation rather than a new theorem. The algorithm maintains an

Table 2: Implementation-level invariants preserved by the framework.

Property	Meaning
Acyclic seed output	LR-TA and the WMSF-style seed both output an acyclic active graph that induces a valid ordering.
Heavy-first add-back	Reinserted edges are processed in nonincreasing original weight, so high-weight arcs are given priority for restoration.
Deterministic topological order	Kahn-style topological reconstruction uses deterministic tie-breaking, making seed output reproducible for fixed input.
Incumbent protection	IPSNS updates the incumbent only when the candidate backward weight is strictly smaller.
Reproducible randomization	IPSNS SCC selection and destroy fractions are fully determined by the configured random seed.

active subgraph over the original edge identifiers and repeatedly applies two phases.

In Phase I, the algorithm searches the active graph for any directed cycle using an iterative depth-first search. The search is deterministic because vertices and outgoing edge lists are scanned in fixed order, and only the first detected cycle is used. If C is the detected cycle, LR-TA computes

$$\varepsilon(C) = \min_{e \in C} w(e) \tag{3}$$

and reduces the mutable weight of every arc in C by $\varepsilon(C)$. At least one arc becomes tight. Any arc whose reduced weight falls below the numerical tolerance is deactivated and appended to the removed-edge set F_{LR}. The phase terminates when no directed cycle remains in the active graph, so the active subgraph is a directed acyclic graph.

In Phase II, LR-TA attempts to reinsert removed arcs without recreating cyclicity. Removed arcs are processed in nonincreasing order of original weight, with deterministic tie-breaking on endpoints and edge identifier. Let π be a topological order of the current active directed acyclic graph and let $\mathrm{rank}_\pi(v)$ denote the position of vertex v. A candidate arc (u, v) is accepted immediately if

$$\mathrm{rank}_\pi(u) < \mathrm{rank}_\pi(v), \tag{4}$$

Algorithm 1 LR-TA seed construction

Require: Weighted digraph $G = (V, A, w)$ with $w \geq 0$
Ensure: Ordering π_{LR}, removed-edge set F_{LR}

Build edge-indexed arrays $(U, V, W_0, W, \mathit{active}, \mathit{adj})$
$F_{\mathrm{LR}} \leftarrow \emptyset$
while the active graph contains a directed cycle C **do**
    $\varepsilon \leftarrow \min_{e \in C} W[e]$
    for all $e \in C$ **do**
        $W[e] \leftarrow W[e] - \varepsilon$
        if $W[e] \leq \tau$ and e is active **then**
            deactivate e; set $W[e] \leftarrow 0$; add e to F_{LR}
        end if
    end for
end while
Compute deterministic topological order π of the active graph
for all $e = (u, v) \in F_{\mathrm{LR}}$ in nonincreasing order of $W_0[e]$ **do**
    if $\mathrm{rank}_\pi(u) < \mathrm{rank}_\pi(v)$ **then**
        reactivate e; remove e from F_{LR}
    else if $v \not\rightsquigarrow u$ in the current active graph **then**
        reactivate e; remove e from F_{LR}
        recompute deterministic topological order π
    end if
end for
return $\pi_{\mathrm{LR}}, F_{\mathrm{LR}}$

because the current topological order already certifies that the insertion is forward. Otherwise the algorithm checks whether v can reach u in the current active graph, using a reachability search pruned by the topological rank interval. The arc is reinserted if and only if no such path exists. Whenever a backward candidate is accepted, the topological order is recomputed so that future admissibility tests remain exact. The extracted topological order is deterministic under the implementation used here, but it is not mathematically unique: different linear extensions of the same DAG can exist. The chosen extraction rule is therefore an engineering convention rather than a canonical ordering of an acyclic state.

Algorithm 1 summarizes the implementation.

The add-back phase is where this implementation departs most clearly

from a bare local-ratio reduction. It is also the component with the strongest ablation signal in the current experiments. LR-TA should therefore be understood as a two-phase engineered seed rather than as a one-step local-ratio routine.

4.2. WMSF-style seed construction

The second seed is a weighted removeArcs–minimize–stabilize pipeline implemented locally as an internal baseline. It broadens the seed space available to the refinement stage and helps define the incumbent-protection guarantee.

At a high level, the WMSF-style seed decomposes the graph into strongly connected components and processes each nontrivial component separately. Within a component, it first removes arcs according to a prescribed weight-based ordering. Two orderings are supported in code: $L1$, which sorts arcs primarily by weight, and $L2$, which normalizes arc weight by a weighted in/out-degree denominator. The resulting removed set is then minimized by heavy-first add-back, followed by a stabilization step and a second minimization pass. When the entire graph is a single nontrivial strongly connected component, both $L1$ and $L2$ are tried and the lower-backward-weight seed is kept. Otherwise the configured ordering is used componentwise.

The point of carrying this seed into IPSNS is not to establish a separate literature claim about WMSF. Instead, it provides a complementary structural bias to LR-TA. LR-TA is driven by cycle reductions in the active graph. The WMSF-style seed is driven by ordered arc removals and subsequent minimization. Because the two constructions fail differently on some instances, taking the better of them before refinement produces a stronger and more robust incumbent than either seed alone.

IPSNS is the main refinement contribution. It combines SCC-local destroy-and-repair moves, restricted local-ratio repair, and restricted heavy-first add-back with a strict incumbent-protection rule. Refinement is therefore concentrated on cyclic regions where ordering uncertainty remains, while the global objective can only improve or stay unchanged.

4.3. Incumbent-protected SCC-neighborhood search

IPSNS begins by computing both seeds on the full graph: the full WMSF-style seed and the LR-TA seed. Their backward weights are evaluated under the common ordering objective, and the better one becomes the initial incumbent. The refinement stage then iterates over strongly connected components of the original aggregated graph, but the selection is filtered by the incumbent

ordering: only components with positive current backward contribution are eligible, and a weighted-random choice is drawn from the top-K contributors by SCC backward weight. This makes the neighborhood search concentrated rather than exhaustive.

Within the chosen component, IPSNS applies a destroy-and-repair move. The destroy stage has two parts. First, a fraction of currently removed internal edges is reactivated in heavy-first order, temporarily giving the neighborhood more freedom. Second, a small fraction of currently active internal edges is forcibly removed in light-first order, encouraging escape from a locally stable order. The repair stage then applies a restricted local-ratio cycle reduction on that SCC neighborhood and finishes with restricted heavy-first add-back minimization. All graph edits are confined to the chosen component's internal edges, while the rest of the incumbent graph remains untouched during the move.

If the repaired candidate fails to admit a valid topological order, the entire SCC-local state is reverted. If the candidate is valid, the full incumbent objective is recomputed. The incumbent is updated only if the new backward weight is strictly smaller than the current best value. Otherwise the move is rejected and the previous incumbent is restored exactly. This is what makes the method incumbent-protected: refinement never commits a worsening move.

Algorithm 2 summarizes the IPSNS driver.

Two implementation details are especially important for interpretation. First, the default WMSF seed mode inside IPSNS is the full per-SCC pipeline, not a weaker legacy approximation; this is required so that the refinement never starts from a seed weaker than the standalone WMSF baseline. Second, the randomization in IPSNS is controlled entirely by a fixed seed. The resulting sequence of SCC choices and destroy fractions is therefore reproducible given the same input and parameter settings. The parameter choices themselves are interpreted conservatively later in the paper: the ablation study supports them as reasonable defaults on the tested subset, not as universally optimal settings.

4.4. Algorithmic invariants

The most important invariant in the framework belongs to IPSNS rather than to LR-TA. Let $\pi^{(0)}$ denote the better of the two internal seeds and let $\pi^{(t)}$ denote the incumbent ordering after t accepted-or-rejected IPSNS

iterations. Because the incumbent is replaced only by a strictly improving candidate, the sequence satisfies

$$\mathrm{bw}\big(\pi^{(t+1)}\big) \le \mathrm{bw}\big(\pi^{(t)}\big) \le \mathrm{bw}\big(\pi^{(0)}\big) \qquad \text{for all } t \ge 0. \tag{5}$$

This monotone non-degradation property is not an approximation guarantee, but it is a concrete algorithmic promise: the final IPSNS solution is never worse than the better internal seed under the same objective and nonnegative-weight assumptions.

LR-TA and the WMSF-style seed have their own feasibility invariants. LR-TA always exits its reduction stage with an acyclic active graph, and its add-back phase only reactivates arcs that preserve acyclicity. The WMSF-style seed similarly alternates removal and minimization steps while checking acyclic feasibility. IPSNS inherits those feasibility properties locally during SCC repair and adds the stronger incumbent-protection rule globally. That combination supports a narrow guarantee about refinement quality without claiming any new theoretical approximation ratio.

5. Experimental design

The experimental design follows a deliberately narrow claim strategy. The study does not attempt to demonstrate that one heuristic dominates every weighted feedback arc set or linear ordering regime. Instead, it evaluates a reproducible framework for nonnegative weighted directed graphs, using sparse benchmark instances as the primary setting and dense complete ordering instances as a transfer test.

Five empirical questions motivate the design. The first is whether the full refinement framework improves on its internal seeds without violating the non-degradation invariant. The second is which components materially contribute to the final solution quality. The third is how close the framework comes to exact optimization on instances where exact optimization is feasible. The fourth is how the framework compares with strong internal and external baselines on the main sparse benchmark. The fifth is how far the sparse-benchmark conclusions transfer to dense complete ordering instances. Table 3 summarizes how the five computational components are used to answer these questions.

5.1. Datasets

The study uses two public benchmark families with different roles in the argument.

Table 3: Overview of the five computational components used in the study.

Study	Purpose	Inst.	Main setting
Internal benchmark	Safety check for LR-TA, WMSF, and IPSNS	105	sparse weighted DIMACS graphs
Ablation study	Add-back, refinement, and iteration study	10	representative sparse subset
Exact validation	Comparison with exact bitmask dynamic programming	57	standard nonnegative instances with $n \leq 20$
Sparse comparison	External baselines on the main benchmark	97	standard nonnegative sparse instances
Dense transfer test	Scope-boundary analysis on LOLIB instances	50	LOLIB complete weighted ordering instances

The primary benchmark family comes from the public `graph-benchmarks` collection Ali Dasdan (2026). After path-level deduplication, this yields 105 unique sparse weighted DIMACS digraph instances. These instances support the internal benchmark, the ablation study, the exact small-instance validation, and the external sparse comparison.

The standard analysis is restricted to the nonnegative-weight subset. Eight instances are excluded from the standard sparse comparison set because they contain negative-weight arcs: `gerez`, `howard-max`, `k3_3`, `ku`, `peterson`, `peterson1`, `peterson2`, and `stg0`. This exclusion is methodological rather than cosmetic. The local-ratio construction and the interpretation of incumbent protection in this study are framed for nonnegative weights, so mixing negative-weight instances into the main tables would weaken the meaning of the guarantee. After exclusions, the primary sparse comparison set contains 97 standard instances.

The exact-validation subset comprises the 57 standard nonnegative benchmark instances with $n \leq 20$ included in the bitmask dynamic-programming study. Negative-weight instances and trivial $n = 0$ cases are excluded. This subset is not treated as a separate benchmark family; it is a validation slice used to judge heuristic quality against certified optima.

The dense transfer test uses a 50-instance subset of the Linear Ordering Problem Library (LOLIB) 2010 Marti et al. (2012); Marti (2010). These

instances differ structurally from the sparse benchmark in a way that matters algorithmically. They are complete weighted ordering instances, and the selected subset spans three families: SGB (25 instances), IO (10 instances), and RandA1 (15 instances across $n = 100$, 150, and 200). In the converted DIMACS representation used in the experiments, every ordered pair carries weight, so these instances are effectively dense complete tournaments or dense linear-ordering inputs rather than sparse general digraphs.

This distinction is central to the scope of the study. The sparse benchmark is the main source of positive claims. LOLIB is not treated as coequal evidence for the primary contribution; it is used to determine where a sparse weighted-digraph heuristic remains competitive and where a dense-ordering-native ordering method becomes preferable.

5.2. Algorithms and baselines

The evaluation includes the proposed refinement framework, its internal seed methods, an exact reference method for the small-instance study, and a set of external or simple baseline heuristics.

The full method of interest is IPSNS, which starts from the better of two internal seeds and applies incumbent-protected SCC-local refinement. The two seeds are LR-TA and WMSF. LR-TA is the engineered local-ratio plus topological add-back construction described in Section 4; WMSF is a weighted removeArcs–minimize–stabilize seed that is carried forward as a baseline and seed source.

For the exact small-instance validation, the reference method is exact bitmask dynamic programming. That solver is not used as a scalable comparator on the full sparse benchmark; its role is only to certify optimality on the small validation subset.

The sparse and dense comparison studies also include five additional baselines:

- the DRMacIver/FAS tool DRMacIver (2026), an external deterministic matrix-based ordering heuristic that maximizes forward weight on pairwise evidence and is evaluated here under the common backward-weight objective;
- python-igraph's Eades-style feedback arc interface python-igraph developers (2026);

- a weighted Eades adaptation implemented locally and described explicitly as an adaptation rather than as the original Eades et al. (1993) algorithm;
- Borda net-score ordering Coppersmith et al. (2010);
- random multistart ranking.

DRMacIver/FAS is not an exact method. Its matrix-based formulation makes it a particularly relevant comparator for dense LOLIB instances, which are complete weighted ordering problems, whereas the sparse benchmark exercises it only through a sparse entry-list interface.

This baseline mix is deliberate. It does not attempt to exhaust every recent ranking model or every exact solver. In particular, it does not include a learned model such as GNNRank He et al. (2022), a memetic linear-ordering solver such as LOP_MA-EDM Carlos Segura (2026), or the recent minimal-and-stable weighted feedback arc set heuristic of Cavallaro and Cutello Cavallaro and Cutello (2025). Those omissions are stated explicitly. The intended comparison surface is the ecosystem of deterministic or near-deterministic weighted-digraph heuristics and accessible external tools that a practitioner could reasonably invoke without building a separate learning or solver pipeline.

Several fairness rules govern the comparisons. First, all quality comparisons use the same backward-weight objective on the original nonnegative arc weights. Second, official external methods and local adaptations are distinguished explicitly. The weighted Eades method is an adaptation of the unweighted Eades idea Eades et al. (1993); it is not presented as an official external implementation. Third, DRMacIver/FAS is reported exactly as it was run in the study. The sparse comparison therefore retains its four incompletions on the standard set, and the dense comparison retains its strong overall performance. Finally, outcomes are interpreted in scope: losing to a dense-ordering-native method on dense LOLIB is not treated as evidence against the sparse-graph claim, and strong sparse performance is not used to imply dense linear-ordering dominance.

5.3. Evaluation metrics

The primary objective throughout the paper is backward weight (BW), defined in Section 3. Lower BW is always better, and all main tables are organized around that quantity.

Several secondary metrics support interpretation. The number of global-best instances records how often a method attains the minimum observed BW among the compared algorithms on a given benchmark. For the external sparse comparison, relative excess over IPSNS records the mean percentage by which a method's BW exceeds IPSNS on completed instances. For the exact small-instance study, the mean gap from the certified optimum and the number of optimal matches summarize how close each heuristic comes to exact optimization. Mean runtime is reported for practical context, although it is not normalized across machines. The dense complete-ordering study also reports forward ratio as a secondary descriptive measure of how much pairwise weight is oriented forward by the returned ranking. Finally, the incumbent-violation count records how often IPSNS would return a solution worse than LR-TA or WMSF; under the intended invariant this count should remain zero on the nonnegative setting. To supplement tabular comparisons, paired statistical tests are computed on the common completed instances of the external sparse comparison, using per-instance backward weight as the outcome unit. The Wilcoxon signed-rank test and sign test characterize the distribution of per-instance paired differences; they are reported as distributional evidence for the observed outcome asymmetry and do not replace exact optimality certificates. A supplementary budget experiment (EXP6) runs IPSNS at six iteration budgets (10, 25, 50, 100, 200, and 400) on a 20-instance representative sparse subset to characterize how solution quality varies with the iteration budget relative to runtime cost.

5.4. Implementation and reproducibility

All reported numbers are taken from saved experimental summaries and regenerated paper assets. Tables and figures in the results section are produced from the recorded JSON, CSV, and Markdown outputs of the five computational components, with key reported numbers checked against their source files before inclusion.

The experimental runs use fixed seeds where randomness exists and consistent wrappers for the compared methods. For IPSNS, the saved summaries record the default full WMSF seed mode and the iteration budgets used in each study. The ablation study is especially important here because it justifies those defaults conservatively: on the 10-instance subset, the mean BW for 50 iterations is identical to the 200-iteration mean, and the no-SCC-priority variant is nearly identical to the default. Those results are interpreted as

subset-level support for reasonable defaults, not as universal parameter optimality.

Negative-weight sparse instances are excluded from the standard claim-bearing benchmark because they do not match the nonnegative setting studied here. DRMacIver/FAS incompletions on the sparse benchmark are reported explicitly rather than hidden in filtered tables. The dense LOLIB transfer test is retained even though it narrows the claim surface, because it documents where the framework remains competitive and where dense complete ordering instances favor a specialized external method.

The review version is prepared under double-anonymized constraints, so code-availability language remains anonymized in the declarations. That packaging choice does not affect the experimental evidence summarized here.

6. Computational results

The computational evidence should be read in the same order as the experimental design. The paper first establishes the main sparse-benchmark result, then checks exact small-instance quality, then uses ablations to interpret the engineering choices, and finally tests transfer to dense LOLIB instances. This ordering matters because the positive claim is intentionally narrow: IPSNS is presented as a strong, reproducible heuristic framework for nonnegative sparse weighted directed graphs, not as a universal best method for every ordering benchmark.

6.1. Sparse weighted graph benchmarks

The primary result comes from the standard nonnegative sparse benchmark, with the full internal benchmark supplying the safety check for the refinement stage. Table 4 summarizes the main comparison on 97 instances, and Figures 2 and 3 visualize the gap structure and the distribution of instance wins.

The main outcome is clear on this benchmark. IPSNS achieves the lowest observed backward weight among the tested methods on 96 of the 97 standard instances. Its mean BW is 37,698, compared with 38,327 for LR-TA and 40,005 for WMSF. Among the external methods, DRMacIver/FAS is the strongest comparator, but its mean BW is still 53,173 on the completed standard instances, or about 21.61% above IPSNS. DRMacIver/FAS completes 93 of 97 instances; the remaining four cases are two DAG inputs rejected by the wrapper and two large sparse instances that time out in the wrapper

Table 4: Comparison on the 97 standard nonnegative sparse instances. BW denotes backward weight; lower values are better. Relative excess is the mean percentage by which an algorithm's BW exceeds IPSNS on completed instances.

Method	Complete	Mean BW	Mean RT (s)	Best	Relative excess
IPSNS (ours)	97/97	37,698	21.92	96	0.00%
LR-TA (ours)	97/97	38,327	0.08	80	0.71%
WMSF seed	97/97	40,005	1.31	61	2.06%
DRMacIver/FAS	93/97	53,173	4.00	56	21.61%
igraph Eades	97/97	95,920	0.01	40	30.54%
Weighted Eades	97/97	99,689	0.11	42	30.48%
Borda net score	97/97	512,277	0.00	27	55.55%
Random multistart	97/97	1,075,258	0.03	42	49.76%

used here. Sparse-comparison averages for DRMacIver/FAS are computed on completed runs only; incompletions are reported separately and are not hidden. The remaining baselines are substantially weaker on average, with the Eades variants about 30% higher than IPSNS and the simpler Borda and random baselines far behind.

This result addresses baseline strength directly. The sparse-benchmark claim is not based only on a comparison among internal variants. It survives contact with a strong external comparator, a widely available graph-library heuristic, and several simple but informative ranking baselines. The most important external comparison is DRMacIver/FAS, not because it wins the benchmark, but because it remains competitive on much of the sparse set despite being a matrix-based pairwise-ordering heuristic rather than a sparse-digraph-native method. Even there, the gap is small only on one instance: `r20_60`, where DRMacIver/FAS beats IPSNS by 3 BW units. That same instance is also the only IPSNS near-miss in the exact-validation study below, which makes the sparse story internally consistent rather than contradictory.

Paired statistical tests on the 101 instances completed by both IPSNS and DRMacIver/FAS show that the per-instance backward-weight differences favor IPSNS asymmetrically: IPSNS achieves a strictly lower backward weight on 37 instances, ties on 56, and is exceeded on 8. Both the Wilcoxon signed-rank test and the sign test yield $p < 0.001$, indicating that the observed win–loss asymmetry is unlikely under a null distribution of symmetric per-instance outcomes. These tests provide distributional support for the benchmark differences; they do not certify optimality. On the full 105-instance internal set,

IPSNS never loses to LR-TA (16 wins, 89 ties, zero losses) and never loses to WMSF (36 wins, 69 ties, zero losses), consistent with the non-degradation invariant. Table 5 summarizes the paired outcomes for all comparisons.

Table 5: Paired statistical tests comparing IPSNS against each baseline on common completed instances of the standard sparse benchmark. Improvement is the per-instance reduction in backward weight achieved by IPSNS (positive values indicate IPSNS attains a lower backward weight). The Wilcoxon signed-rank test and sign test are two-sided and provide distributional support for the observed differences; they do not replace exact optimality certificates.

Comparison	n	W	T	L	Med. ΔBW	p (Wilcoxon / Sign)
IPSNS vs LR-TA	105	16	89	0	0	<0.001 / <0.001
IPSNS vs WMSF	105	36	69	0	0	<0.001 / <0.001
IPSNS vs DRMacIver/FAS	101	37	56	8	0	<0.001 / <0.001
IPSNS vs igraph Eades	105	59	41	5	92	<0.001 / <0.001
IPSNS vs Weighted Eades	97	55	42	0	859	<0.001 / <0.001

W = wins (IPSNS lower BW), T = ties, L = losses. Med. ΔBW is the per-instance median improvement in backward-weight units. DRMacIver/FAS has 101 paired instances (four sparse incompletions excluded). IPSNS never loses to LR-TA or WMSF (zero losses), consistent with the non-degradation invariant.

The full internal benchmark strengthens this interpretation by showing that the refinement stage does not obtain these gains by occasionally damaging its seeds. Across 105 sparse benchmark instances, IPSNS is never worse than LR-TA and never worse than WMSF, with zero incumbent-protection violations in the verified summary. It strictly improves over LR-TA on 16 instances and over WMSF on 36 instances. The mean runtime per instance rises from 0.074 seconds for LR-TA and 1.24 seconds for WMSF to about 20.2 seconds for IPSNS, so the gain is not free; however, it is gained without giving up the best internal seed. That property connects the empirical behavior back to the monotone non-degradation invariant in Section 4.

The runtime pattern also clarifies the engineering tradeoff. LR-TA is the fastest strong method in the study: its mean sparse-benchmark runtime is only 0.08 seconds per instance, yet its mean BW is within about 1.7% of IPSNS. That makes LR-TA a credible low-latency choice when refinement time is limited. IPSNS then occupies the higher-quality point on the tradeoff curve, paying more runtime to secure the best observed solutions on almost every instance. The study therefore provides a fast seed heuristic, a stronger refinement framework, and a clear statement of what the extra computation

buys.

A supplementary budget study (EXP6) tests how IPSNS quality varies with the iteration budget on a 20-instance representative sparse subset (instances with EXP4 IPSNS runtime exceeding 60 s or fewer than 10 vertices excluded). Across six budgets from 10 to 400 iterations, the mean backward weight on this subset is identical (84,614 BW units) and no losses to LR-TA occur at any budget. The improvement over LR-TA is concentrated on 7 of the 20 instances and is secured within the first 10 iterations; additional budget does not change the outcome on this subset. Runtime scales approximately linearly: mean per-instance time rises from 0.28 s at 10 iterations to 4.40 s at 400 iterations. Table 6 and Figure 4 summarize these results.

Table 6: IPSNS quality-vs-runtime tradeoff across iteration budgets on a 20-instance representative sparse subset (EXP6). Mean backward weight (BW), mean runtime per instance, and win/tie/loss counts versus LR-TA from EXP4 are shown for each budget. Rel. excess is the mean percentage by which that budget's BW exceeds the 400-iteration result on the same subset; lower is better. LR-TA (from EXP4) is included as the low-latency reference.

Algorithm (budget)	Mean BW	Mean RT (s)	W / T / L vs LR-TA	Rel. excess vs 400-iter
LR-TA (seed only)	87,365	0.035	— / — / —	—
IPSNS (10 iters)	84,614	0.275	7 / 13 / 0	0.00%
IPSNS (25 iters)	84,614	0.425	7 / 13 / 0	0.00%
IPSNS (50 iters)	84,614	0.698	7 / 13 / 0	0.00%
IPSNS (100 iters)	84,614	1.222	7 / 13 / 0	0.00%
IPSNS (200 iters)	84,614	2.286	7 / 13 / 0	0.00%
IPSNS (400 iters)	84,614	4.398	7 / 13 / 0	0.00%

Subset: 20 representative sparse instances with EXP4 IPSNS runtime ≤ 60 s and $n \geq 10$, spanning density from very sparse to moderately dense. W = wins (IPSNS lower BW than LR-TA), T = ties, L = losses (none observed). Rel. excess: mean percentage above the 400-iteration result on the same instance set.

The sparse results cover a broad but intentionally scoped benchmark. The comparison uses 97 standard nonnegative sparse instances after exclusions and includes eight algorithms in total. The claim remains limited to this setting and does not extend to negative-weight cases or dense complete ordering benchmarks.

6.2. Exact validation and ablation evidence

The exact small-instance validation and the ablation study serve different purposes, but together they explain why the framework is both credible and interpretable. Table 7 reports the exact validation results, and Table 8 reports the component ablation.

Table 7: Exact validation on the 57 standard nonnegative instances with $n \leq 20$ included in the dynamic-programming study. Mean gap is measured against the certified optimum on each instance.

Method	Optimal	Optimality rate	Mean gap
IPSNS (ours)	56/57	98.2%	0.0006%
LR-TA (ours)	55/57	96.5%	0.0590%
WMSF seed	51/57	89.5%	0.0961%

Table 8: Ablation study on 10 representative sparse instances. Relative change is measured against LR-TA; negative percentages indicate improvement.

Variant	Mean BW	Relative change	Mean RT (s)
LR without add-back	4525.1	+5.94%	0.017
LR-TA	4271.5	+0.00%	0.047
WMSF seed	4332.5	+1.43%	0.040
Best seed, no refinement	4271.5	+0.00%	0.069
IPSNS, 50 iterations	4239.2	-0.76%	0.778
IPSNS, 200 iterations	4239.2	-0.76%	5.734
IPSNS, no SCC priority	4239.1	-0.76%	5.727

The exact small-instance study supports a quality-validation narrative rather than a theoretical one. On the 57 standard nonnegative instances with $n \leq 20$ included in the dynamic-programming study, IPSNS matches the certified optimum on 56 instances and has a mean gap of 0.0006% from the exact optimum. LR-TA matches the optimum on 55 instances with mean gap 0.0590%, and WMSF matches the optimum on 51 with mean gap 0.0961%. These numbers show that the framework is not merely competitive in relative benchmark terms; on the exact-validation subset, it is essentially exact in practice.

This table does not provide an approximation guarantee, a worst-case bound, or a proof of global behavior beyond the validated subset. The correct statement is that IPSNS is empirically near-optimal on the exact-validation subset. The only IPSNS near-miss is instance `r20_60`.

The exact-validation result also helps interpret the single sparse-benchmark loss to DRMacIver/FAS. The only IPSNS near-miss is again `r20_60`. This is also the only standard small instance where IPSNS fails to

match exact optimum. That consistency reduces the chance that the sparse comparison is driven by unstable reporting or artifact mismatch. Instead, it suggests a small, structurally difficult case where the framework is near optimum but not uniquely dominant.

The ablation study explains where the framework's gains come from. The add-back phase is the dominant contributor among the tested design choices. Removing it raises mean BW from 4,271.5 for LR-TA to 4,525.1 for the no-add-back variant, a degradation of about 5.94%. This is the clearest experimental support for treating topological add-back as a substantive engineering contribution rather than a decorative post-processing step.

The refinement stage then provides a smaller but still measurable gain. Moving from LR-TA to IPSNS with 50 iterations reduces mean BW from 4,271.5 to 4,239.2, about 0.76%. In absolute terms that gain is much smaller than the add-back gain, but it is still meaningful because it is obtained after starting from a strong seed and while preserving the non-degradation invariant. This is the kind of marginal improvement one expects from a bounded refinement layer: it does not re-invent the seed; it improves the hard residue that remains after seeding.

The ablation results also provide the main evidence for parameter justification, and the interpretation must remain conservative. On this 10-instance subset, the add-back phase accounts for the largest mean improvement (about 5.9%), while IPSNS contributes a further 0.75%. The 50-iteration and 200-iteration IPSNS variants have the same mean BW, which suggests that the default budget is already beyond the point of visible average improvement on these instances. Likewise, removing SCC priority changes the mean only from 4,239.2 to 4,239.1, effectively negligible at the reported precision. These observations support the default settings as reasonable choices, but they do not show that those settings are universally optimal on other graph families or scales.

6.3. Dense LOLIB transfer test

Table 9 and Figure 5 summarize the dense LOLIB transfer test.

This is the result that prevents overclaiming. On the 50 dense complete ordering instances, DRMacIver/FAS is the best overall method in the study: it wins 45 instances and achieves mean BW 571,687, compared with 582,354 for IPSNS. The mean gap is 3.88% in DRMacIver/FAS's favor. IPSNS remains second overall and retains zero incumbent violations against LR-TA

Table 9: Dense LOLIB transfer test on 50 complete weighted ordering instances. Lower BW is better.

Method	Mean BW	Best	Mean RT (s)
DRMacIver/FAS	571,687	45/50	1.01
IPSNS (ours)	582,354	5/50	37.92
LR-TA (ours)	582,948	2/50	0.22
WMSF seed	585,926	1/50	0.19
igraph Eades	659,570	0/50	0.01
Weighted Eades	675,235	0/50	0.01
Random multistart	883,664	0/50	0.02
Borda net score	1,066,045	0/50	0.00
Family breakdown for IPSNS and DRMacIver/FAS			
Family	IPSNS best	DRMacIver/FAS best	Mean BW (IPSNS / DRMacIver/FAS)
SGB	1/25	24/25	1,048,056 / 1,036,085
IO	4/10	6/10	36,996 / 32,860
RandA1	0/15	15/15	169,755 / 156,909

and WMSF, but the dense benchmark does not support any claim of overall dominance by the proposed framework.

The family breakdown explains why. DRMacIver/FAS wins 24 of the 25 SGB instances and all 15 RandA1 instances. IPSNS is more competitive on IO, where it wins 4 of 10 against DRMacIver/FAS's 6 of 10, but that does not overturn the overall dense-ordering result. The cleanest interpretation is structural: DRMacIver/FAS is a matrix-based pairwise-ordering heuristic whose documented input model aligns with LOLIB's complete weighted ordering format, whereas IPSNS is designed around sparse weighted-digraph seeding and SCC-local refinement. Dense complete ordering remains an active research area Fomin et al. (2010); Ostovari and Zarei (2024), and when every ordered pair carries weight the advantage shifts toward methods specialized for matrix-based pairwise evidence.

This finding narrows the claim in a credible way. Retaining the dense transfer study shows that the framework transfers nontrivially to a dense benchmark, remains competitive on a structured IO subset, and still preserves the internal no-worsening invariant, but dense complete ordering is not the framework's strongest regime.

Across the section as a whole, the empirical message is coherent. The strongest claim belongs to the sparse nonnegative benchmark, where IPSNS attains the best observed backward weight among the tested methods on 96 of 97 standard instances and never returns a result worse than the better

internal seed. The exact-validation subset shows that the quality story is not merely relative to heuristics, and the ablation study shows that the gains are not coming from an uninterpretable black box. The dense transfer study then marks the boundary of that success case. The framework is a strong and reproducible method for the sparse weighted-digraph regime, and the computational evidence shows how far that statement can be pushed.

7. Discussion

The computational evidence supports a clear but bounded interpretation of the proposed framework. The strongest result is the sparse nonnegative benchmark, where IPSNS achieves the lowest observed backward weight among the tested methods on 96 of 97 standard instances and never degrades the better internal seed. The importance of that result is not only that the framework performs well, but that it performs well in a way that matches the intended algorithmic design. The methodology combines two complementary global constructions with a refinement stage that concentrates effort on the cyclic parts of the graph and accepts a move only when the global objective improves. The sparse results therefore do not look accidental: they are consistent with the structure that the method is designed to exploit.

The sparse benchmark interpretation begins with the role of the seed methods. LR-TA and WMSF produce feasible orderings by different mechanisms. LR-TA is driven by local-ratio reductions and topological add-back, whereas WMSF follows a removal-and-minimization template. On many sparse instances those two constructions are already strong, and LR-TA in particular offers an excellent quality–runtime tradeoff. IPSNS improves on that base by focusing its neighborhood moves on strongly connected components that still contribute substantial backward weight under the incumbent ordering. This is a natural strategy for sparse cyclic graphs. When the graph contains large acyclic regions and a smaller cyclic core, indiscriminate perturbation is wasteful; the search budget is better spent where ordering ambiguity remains concentrated. The empirical advantage of IPSNS on the sparse benchmark is therefore not simply that it runs longer than LR-TA. It is that the extra computation is directed at the right structural target.

Structural diagnostics computed from the converted DIMACS instance files confirm this contrast quantitatively. The standard sparse benchmark instances have a mean directed-graph density of 0.354 and a mean of 72.7% of vertices residing in nontrivial strongly connected components, with the SCC

structure distributed across multiple components on most instances. The dense LOLIB instances, by contrast, have a mean density of 0.454 and essentially all vertices (98.7% on average) concentrated in a single large SCC, which is consistent with the complete pairwise-ordering matrix structure. On the sparse benchmark, the IPSNS advantage over DRMacIver/FAS is negatively correlated with instance density (Spearman $r = -0.76$, $p < 0.001$), consistent with the expectation that matrix-based pairwise-ordering methods become more competitive as graph density increases toward the complete-ordering regime. These diagnostics are consistent with the intended operating regime of IPSNS; they do not replace controlled structural experiments. Table 10 summarizes the structural comparison.

Table 10: Structural comparison between the standard sparse benchmark and the dense LOLIB transfer benchmark. Density is $m/n(n-1)$ for directed graphs. Largest-SCC fraction is the fraction of vertices belonging to the largest strongly connected component. All statistics are computed from the converted DIMACS instance files used in the experiments.

Feature	Sparse benchmark	Dense LOLIB
Instances	103	50
Mean n (vertices)	802	94
Mean m (arcs)	1404	5083
Mean density	0.3535	0.4542
Mean largest-SCC frac.	0.6018	0.9867
Mean frac. in nontrivial SCCs	0.7271	0.9867
Mean nontrivial SCC count	10.3	1.0

Sparse benchmark uses the `graph-benchmarks` collection (standard 97-instance nonnegative subset after exclusions). LOLIB dense benchmark uses 50 converted DIMACS instances across three families (SGB, RandA1, IO). SCC features computed from converted DIMACS instance files.

The internal safety result is central to this interpretation. On the full 105-instance internal benchmark, IPSNS never returns a solution worse than LR-TA or WMSF. That is more than a convenient side condition. In practical decision-support or ordering pipelines, a refinement stage that occasionally damages a good seed is hard to trust, hard to tune, and difficult to diagnose. The monotone non-degradation rule makes the framework traceable. If refinement finds no helpful move, the best seed is preserved. If refinement improves the incumbent, that improvement is directly visible in the final objective. For an engineering-oriented audience, this kind of operational reliability is often more useful than a loosely justified claim that a local search

method is "usually better."

The exact-validation subset strengthens the quality story without changing its theoretical status. IPSNS matches the certified optimum on 56 of 57 standard nonnegative instances in the validation subset and has a mean gap of 0.0006%. That is strong evidence that the heuristic framework is solving the small instances almost perfectly in practice. It is not, however, evidence for a new approximation ratio or a new optimality guarantee beyond the validated subset. When exact optimization is feasible, the framework tracks the optimum closely; when the instances become larger, the study relies on comparative benchmark evidence rather than on a theoretical claim it does not possess.

The ablation study helps clarify which parts of the framework are carrying the result. The topological add-back phase is the dominant design element among the tested components, with a mean backward-weight reduction of about 5.9% relative to local-ratio without add-back. That is the main reason LR-TA should be understood as an engineered instantiation rather than as a thin wrapper around prior local-ratio ideas. The refinement stage then contributes a smaller additional improvement, about 0.75% on the ablation subset. Seed construction determines most of the ordering quality; refinement cleans up the remaining difficult cases without sacrificing the incumbent. On the full 105-instance sparse set, IPSNS strictly improves over LR-TA on 16 instances (mean improvement 581 BW units; median 0, since most instances are already tied at the seed quality), confirming that the gains are concentrated on a harder subset rather than distributed uniformly; the runtime cost is the approximately 270-fold increase from LR-TA's 0.08 s mean to IPSNS's 21.9 s mean per instance. The supplementary budget study (EXP6) on a 20-instance representative sparse subset confirms that quality saturates within the first 10 SCC-local iterations on that subset; additional budget increases runtime proportionally without improving the mean backward weight, suggesting that the targeted SCC-local moves are effective from the earliest iterations rather than requiring prolonged search. The observed near-equality between 50 and 200 IPSNS iterations on the ablation subset also clarifies how the parameter choices should be discussed. They appear reasonable and stable on the tested subset, but they should not be promoted as universally optimal defaults for every future graph family.

The dense LOLIB transfer test is the key scope boundary. On that benchmark, DRMacIver/FAS is stronger overall, winning 45 of 50 instances and outperforming IPSNS in mean backward weight. This result is part of the

scientific contribution because it shows where the framework stops being the most competitive option among the methods tested. The structural explanation is straightforward. DRMacIver/FAS is a deterministic matrix-based pairwise-ordering heuristic, and LOLIB supplies dense complete ordering instances where every ordered pair contributes information. IPSNS, by contrast, is built around sparse weighted-digraph seeding and SCC-local refinement. Recent dense-ordering feedback arc set methods Fomin et al. (2010); Ostovari and Zarei (2024) reinforce that complete weighted ordering is a distinct algorithmic regime rather than a mere scale change from sparse digraphs. When the graph is fully dense, the distinction between cyclic core and acyclic background becomes much less informative, and the advantage shifts toward methods specialized for matrix-based pairwise evidence.

At the same time, the dense result should not be overinterpreted in the opposite direction. IPSNS remains second overall on LOLIB, preserves its non-degradation property, and is competitive on the IO family, where it wins 4 of 10 instances. Dense complete ordering is not the paper's primary target, and a matrix-based pairwise-ordering heuristic is preferable there. Identifying where the methodology is strong, where it is merely competitive, and where a different structural assumption favors another class of methods improves the credibility of the overall claim.

Several limitations remain and should be stated directly. First, the paper does not offer a new approximation guarantee. The local-ratio principle used in LR-TA is prior art, and the present contribution is an engineered reproducible instantiation plus the incumbent-protected SCC refinement framework. Second, the main claims apply only to nonnegative-weight instances. Negative-weight sparse instances are excluded from the standard analysis because they do not match the intended setting of the framework and would blur the meaning of the reported invariant. Third, the exact-validation study is limited to small instances where bitmask dynamic programming is feasible. It supports the quality narrative but does not remove the need for heuristic benchmarking on larger graphs. Fourth, dense linear-ordering instances are not the primary target, and the LOLIB results show that clearly. Fifth, the review version of the paper is anonymized, so data and code availability must be described in double-blind form until the submission process allows a public release.

These limitations also help identify the practical implications. The paper is best read as a contribution to weighted ordering and cyclic-dependency repair in graph-based decision-support settings. Many engineering and opera-

tions workflows contain precedence conflicts, pairwise preference information, or cyclic dependencies that must be resolved into an ordering without claiming that every arc or relation can be simultaneously respected. In such settings, a methodology that starts from strong fast seeds, improves only when the objective truly decreases, and remains reproducible under fixed seeds and explicit tie handling can be more valuable than a theoretically stronger method that is harder to deploy or interpret. The sparse benchmark results suggest that the framework is well suited to that style of application: graphs are large enough to make exact optimization impractical, sparse enough that SCC-local refinement remains meaningful, and structured enough that the combination of LR-TA, WMSF, and IPSNS improves over simpler ranking heuristics.

Future work should follow from those boundaries rather than ignore them. One direction is richer parameter tuning, especially to understand when longer refinement budgets or alternative SCC-selection rules materially improve sparse instances beyond the current defaults. Another is broader evaluation on ranking and application-facing datasets that better expose domain structure than generic benchmarks alone. A third is exact or ILP-based polishing on medium-sized instances if solver setup becomes available in a later study. That would not replace the present heuristic contribution, but it could sharpen the quality reference point between the small exact-validation regime and the large sparse benchmark regime. A fourth is broader study of negative-weight or mixed-sign settings under a formulation that preserves a clean interpretation of the objective and the refinement invariant.

Overall, the study is strongest when it remains faithful to its empirical boundaries. It presents a reproducible methodology for the minimum weighted feedback arc set problem in nonnegative weighted directed graphs, demonstrates clear strength on the main sparse benchmark, and documents where a different structural regime favors a different algorithmic family.

8. Conclusions

This paper studied the minimum weighted feedback arc set problem through its ordering interpretation on nonnegative weighted directed graphs. The main contribution is not a new approximation framework, but a reproducible computational methodology that combines an engineered local-ratio construction with topological add-back, a complementary weighted seed, and an incumbent-protected SCC-local refinement stage. Within that design, LR-

TA serves as a strong constructive seed, while IPSNS is the main refinement framework that improves the incumbent only when the global backward-weight objective decreases.

The computational evidence supports a clear scope-bounded conclusion. On the standard nonnegative sparse benchmark, IPSNS attains the best observed backward weight among the tested methods on 96 of 97 instances and improves on all tested internal and external comparators, including the strongest external baseline. On the full internal benchmark, the refinement stage never worsens the better internal seed, which confirms the intended non-degradation behavior in practice.

The exact-validation subset shows that the framework is near-optimal when bitmask dynamic programming certifies optima. The ablation study shows that the topological add-back phase is the dominant design element, with SCC-local refinement providing an additional but smaller gain.

The results also define an important boundary. On dense complete LOLIB ordering instances, the matrix-based pairwise-ordering baseline DR-MacIver/FAS is stronger overall. This does not negate the sparse-benchmark contribution; it clarifies it. The proposed framework is strongest in the sparse nonnegative weighted-digraph regime for which it was designed. It should be interpreted as a general weighted-digraph heuristic rather than as a dense-native linear-ordering solver.

The practical value of the framework lies in this combination of quality, structural focus, and reproducibility. The methodology is suitable for weighted ordering and cyclic-dependency repair settings where exact optimization is too expensive, sparse graph structure is meaningful, and refinement should not be allowed to damage a good seed. Within the stated boundaries, engineered local-ratio seeding plus incumbent-protected SCC refinement is an effective strategy for sparse weighted directed ordering problems.

Two immediate directions follow from this evidence. The first is broader evaluation on additional sparse weighted digraph families, especially application-derived instances with richer edge-weight semantics. The second is methodological: the same incumbent-protected refinement discipline could be combined with stronger dense-ordering seeds or exact large-neighborhood subroutines without changing the basic non-degradation philosophy that motivated the present framework.

CRediT authorship contribution statement

Soroush Vahidi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Visualization.

Funding statement

This research did not receive external funding.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Data and code availability

The code, processed result summaries, and manuscript-support files for this study will be released in a public versioned repository and archived package upon acceptance. For double-anonymized review, an anonymous reproducibility artifact is provided separately. Public benchmark instances are available from the sources cited in the manuscript, and the reproducibility package documents the conversion and preprocessing steps used in this study.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used AI-assisted tools, including ChatGPT, Codex, Claude, and Perplexity AI, to support literature exploration, organization of experimental material, and language editing. The authors reviewed and edited all generated content, verified the relevant sources and experimental results, and take full responsibility for the content of the submitted manuscript.

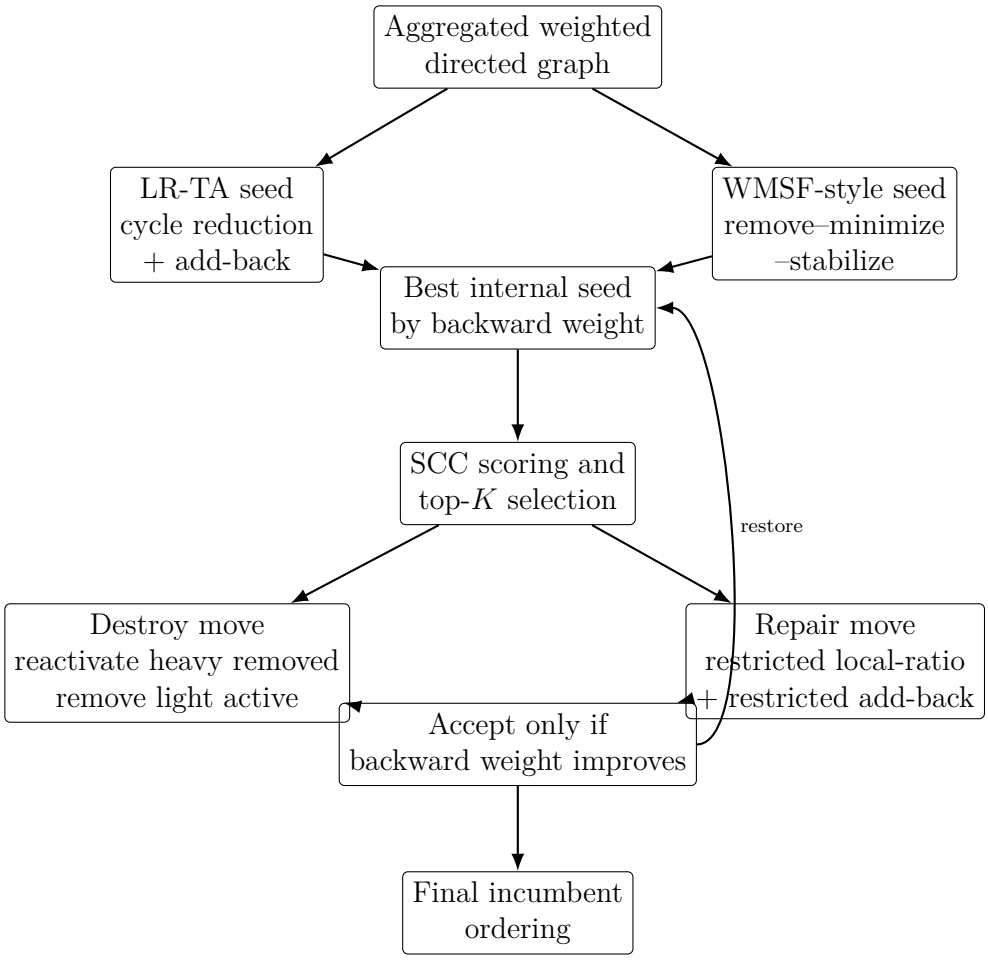

Figure 1: Conceptual overview of the integrated framework. Two constructive seeds are evaluated first; IPSNS then applies incumbent-protected SCC-local destroy-and-repair moves to the better seed only when they improve the global objective.

Algorithm 2 IPSNS with incumbent protection

Require: Weighted digraph $G = (V, A, w)$, iteration budget T
Ensure: Ordering $\pi^\star$ with objective no worse than the best internal seed

Build WMSF-style seed π_{W} and LR-TA seed π_{LR}
$\pi^\star \leftarrow \arg\min\{\mathrm{bw}(\pi_{\mathrm{W}}), \mathrm{bw}(\pi_{\mathrm{LR}})\}$
for $t = 1, \ldots, T$ **do**
    Score each nontrivial SCC by its current backward contribution
    if all SCC scores are zero **then**
        break
    end if
    Sample one SCC from the top-K positive-score SCCs using weighted random choice
    Save the current internal SCC state
    Reactivate a heavy fraction of removed internal edges
    Remove a light fraction of active internal edges
    Apply SCC-restricted local-ratio repair
    Apply SCC-restricted heavy-first add-back minimization
    if the candidate SCC state is invalid **then**
        restore saved SCC state; **continue**
    end if
    Recompute the global ordering and candidate objective
    if the candidate objective is smaller than $\mathrm{bw}(\pi^\star)$ **then**
        accept the candidate and update $\pi^\star$
    else
        restore the previous incumbent exactly
    end if
end for
return $\pi^\star$

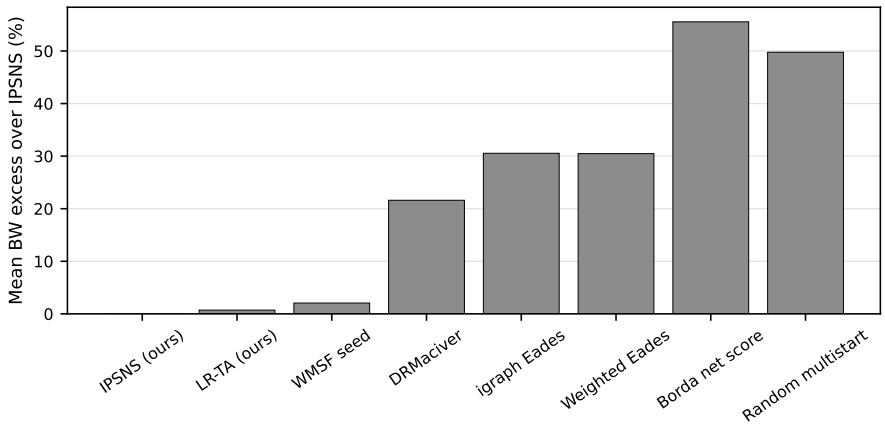

Figure 2: Mean backward-weight excess over IPSNS on the 97 standard nonnegative sparse instances. Lower is better; IPSNS is the reference at zero.

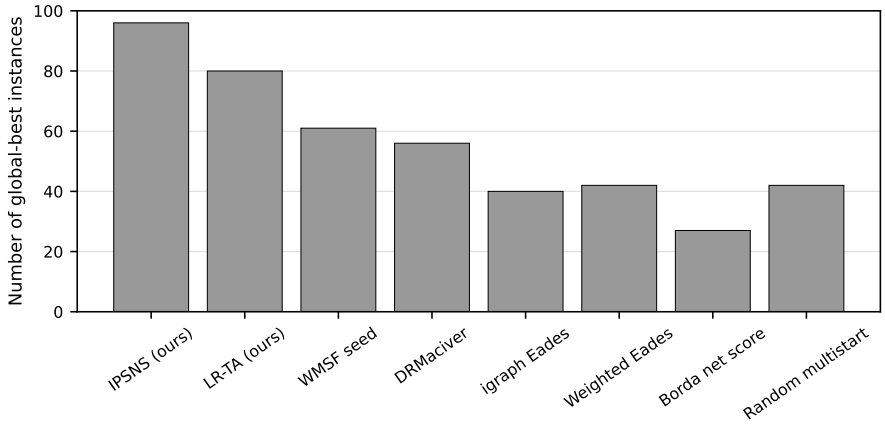

Figure 3: Number of instances on which each method attains the lowest observed backward weight on the primary sparse benchmark.

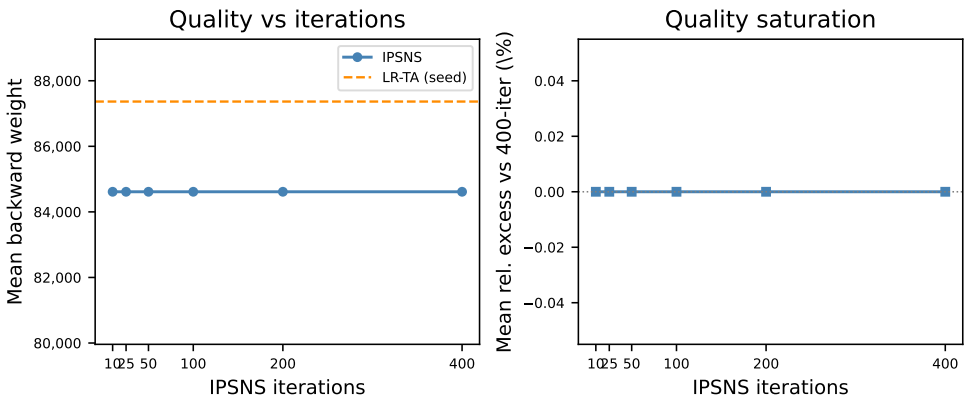

Figure 4: IPSNS quality and saturation on the 20-instance representative sparse subset (EXP6). *Left:* mean backward weight versus iteration budget; LR-TA (dashed) is the low-latency reference. *Right:* mean relative excess above the 400-iteration result; zero at all tested budgets indicates complete quality saturation from the first budget tested on this subset.

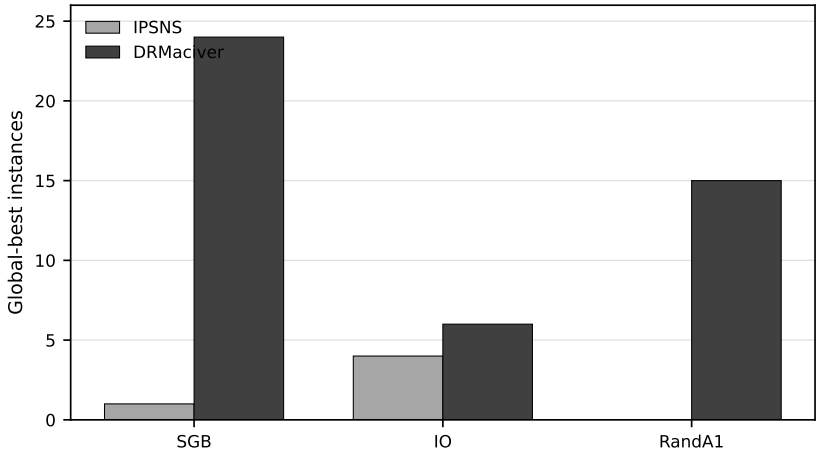

Figure 5: Global-best counts by LOLIB family for IPSNS and DRMacIver/FAS.

References

Ailon, N., Charikar, M., Newman, A., 2008. Aggregating inconsistent information: Ranking and clustering. Journal of the ACM 55, 23:1–23:27. doi:10.1145/1411509.1411513.

Ali Dasdan, 2026. graph-benchmarks: Directed graph benchmarks in dimacs graph format. https://github.com/alidasdan/graph-benchmarks. Dataset repository; accessed 2026-06-06.

Alon, N., Lokshtanov, D., Saurabh, S., 2009. Fast FAST, in: Automata, Languages and Programming, Springer Berlin Heidelberg. pp. 49–58.

Baharev, A., Schichl, H., Neumaier, A., Achterberg, T., 2021. An exact method for the minimum feedback arc set problem. ACM Journal of Experimental Algorithmics 26, 1.4:1–1.4:28. doi:10.1145/3446429.

Bar-Yehuda, R., Geiger, D., Naor, J.S., Roth, R.M., 1998. Approximation algorithms for the feedback vertex set problem with applications to constraint satisfaction and bayesian inference. SIAM Journal on Computing 27, 942–959. doi:10.1137/S0097539796305109.

Carlos Segura, 2026. Lop_ma-edm: Memetic algorithms with explicit diversity management for the linear ordering problem. https://github.com/carlossegurag/LOP_MA-EDM. Software repository; accessed 2026-06-06.

Cavallaro, C., Cutello, V., 2025. An efficient heuristic algorithm to compute minimal and stable weighted feedback arc sets, in: Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE), pp. 84–87. doi:10.18293/SEKE2025-049.

Coppersmith, D., Fleischer, L.K., Rurda, A., 2010. Ordering by weighted number of wins gives a good ranking for weighted tournaments. ACM Transactions on Algorithms 6, 55:1–55:13. doi:10.1145/1798596.1798608.

Demetrescu, C., Finocchi, I., 2003. Combinatorial algorithms for feedback problems in directed graphs. Information Processing Letters 86, 129–136. doi:10.1016/S0020-0190(02)00491-X.

DRMacIver, 2026. Feedback-arc-set. https://github.com/DRMacIver/Feedback-Arc-Set. Software repository; experiment commit 16ff24a92fde886e58819180a9fe686e60991c5c; accessed 2026-06-06.

Eades, P., Lin, X., 1995. A heuristic for the feedback arc set problem. Australasian Journal of Combinatorics 12, 15–25.

Eades, P., Lin, X., Smyth, W.F., 1993. A fast and effective heuristic for the feedback arc set problem. Information Processing Letters 47, 319–323. doi:10.1016/0020-0190(93)90079-O.

Even, G., Naor, J.S., Schieber, B., Sudan, M., 1998. Approximating minimum feedback sets and multicuts in directed graphs. Algorithmica 20, 151–174. doi:10.1007/PL00009191.

Flood, M.M., 1990. Exact and heuristic algorithms for the weighted feedback arc set problem: A special case of the skew-symmetric quadratic assignment problem. Networks 20, 1–23. doi:10.1002/net.3230200102.

Fomin, F.V., Lokshtanov, D., Raman, V., Saurabh, S., 2010. Fast local search algorithm for weighted feedback arc set in tournaments. Proceedings of the AAAI Conference on Artificial Intelligence 24, 65–70. doi:10.1609/aaai.v24i1.7557.

He, Y., Gan, Q., Wipf, D., Reinert, G.D., Yan, J., Cucuringu, M., 2022. Gnnrank: Learning global rankings from pairwise comparisons via directed graph neural networks, in: Proceedings of the 39th International Conference on Machine Learning, pp. 8581–8612. URL: https://proceedings.mlr.press/v162/he22b.html.

Hecht, M., Gonciarz, K., Horvát, S., 2021. Tight localizations of feedback sets. ACM Journal of Experimental Algorithmics 26, 1.3:1–1.3:19. doi:10.1145/3447652.

Karp, R.M., 1972. Reducibility among combinatorial problems, in: Complexity of Computer Computations. Springer US, Boston, MA, pp. 85–103. doi:10.1007/978-1-4684-2001-2_9.

Lempel, A., Cederbaum, I., 1966. Minimum feedback arc and vertex sets of a directed graph. IEEE Transactions on Circuit Theory 13, 399–403. doi:10.1109/TCT.1966.1082620.

Marti, R., 2010. Lolib: Linear ordering problem library. https://www.uv.es/~rmarti/paper/lop.html. Official library page; accessed 2026-06-06.

Marti, R., Reinelt, G., Duarte, A., 2012. A benchmark library and a comparison of heuristic methods for the linear ordering problem. Computational Optimization and Applications 51, 1297–1317. doi:10.1007/s10589-010-9384-9.

Ostovari, M., Zarei, A., 2024. A better LP rounding for feedback arc set on tournaments. Theoretical Computer Science 1015, 114768. doi:10.1016/j.tcs.2024.114768.

python-igraph developers, 2026. python-igraph documentation: Graph.feedback_arc_set. https://python.igraph.org/en/stable/api/igraph.Graph.html. Documentation for the weighted feedback arc set API; accessed 2026-06-06.

Simpson, M., Srinivasan, V., Thomo, A., 2016. Efficient computation of feedback arc set at web-scale. Proceedings of the VLDB Endowment 10, 133–144. doi:10.14778/3021924.3021930.